

Modelagem de Software

Aula 03

Prof. Dr. Rodrigo Plotze

rodrigoplotze@gmail.com

Modelagem de Software

- Conceitos fundamentais
- Tipos de modelagem
- Introdução à UML
- Diagrama de Caso de Uso

MODELAGEM DE SOFTWARE

O que é Modelagem de Software?

Processo de **representar um sistema utilizando modelos abstratos**, facilitando a compreensão, análise e desenvolvimento do software.

Objetivos da modelagem:

- Visualizar o sistema antes da implementação
- Documentar a estrutura e comportamento
- Reduzir complexidade
- Facilitar comunicação entre equipe

Por que modelar?

- Melhor entendimento do sistema
- Identificação de problemas antes da implementação
- Comunicação entre desenvolvedores
- Documentação do projeto
- Facilita manutenção e evolução

O que é um Modelo?

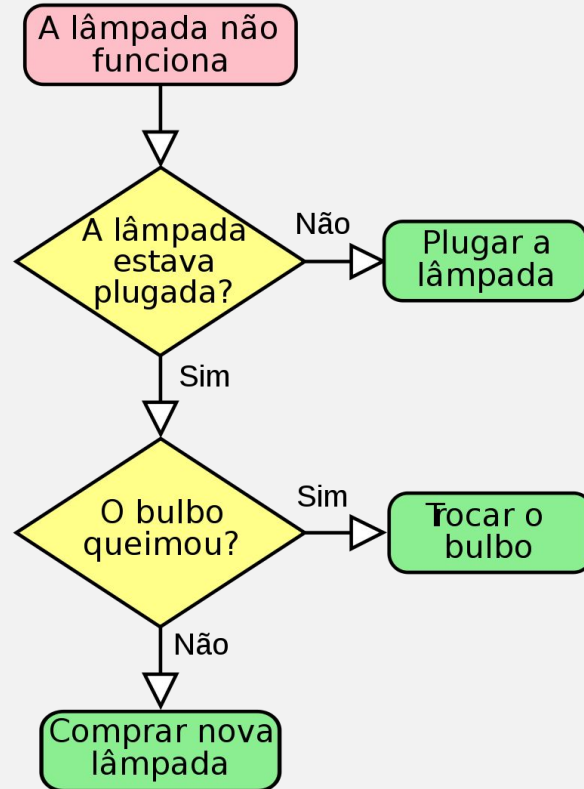
Um **modelo** é uma **representação simplificada da realidade**. Exemplos:

- Planta de uma casa
- Mapa de uma cidade
- Diagrama de um sistema

No projetos de software, modelos representam:

- Estrutura do sistema
- Comportamento
- Interações entre componentes

Fluxograma



- Modelo de Entidade e Relacionamento

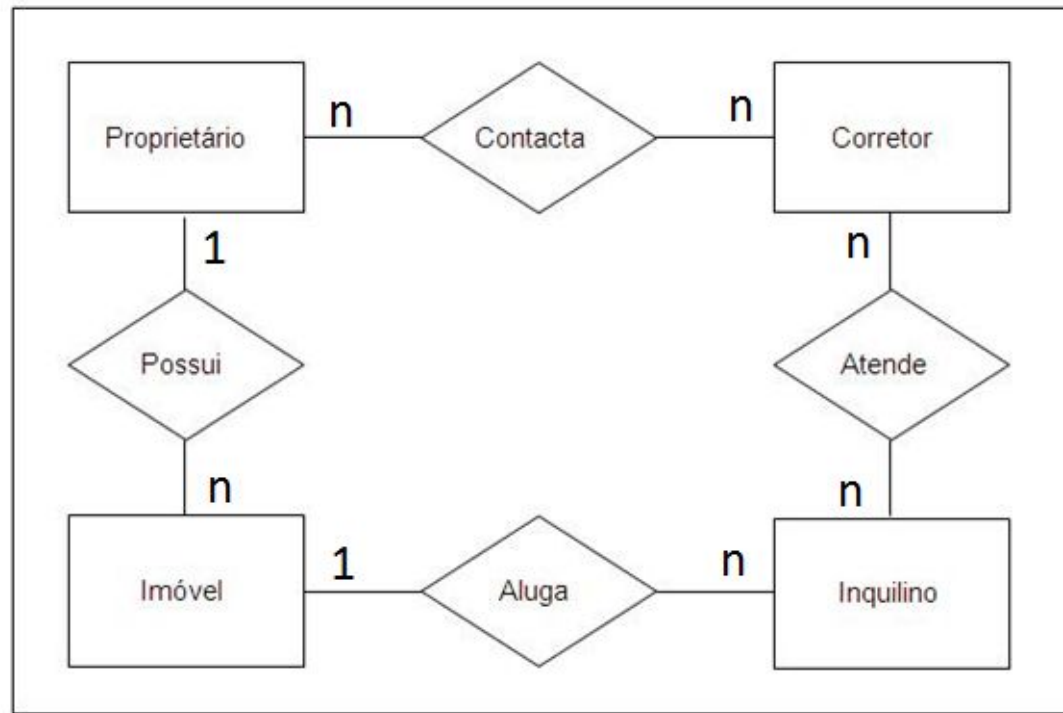
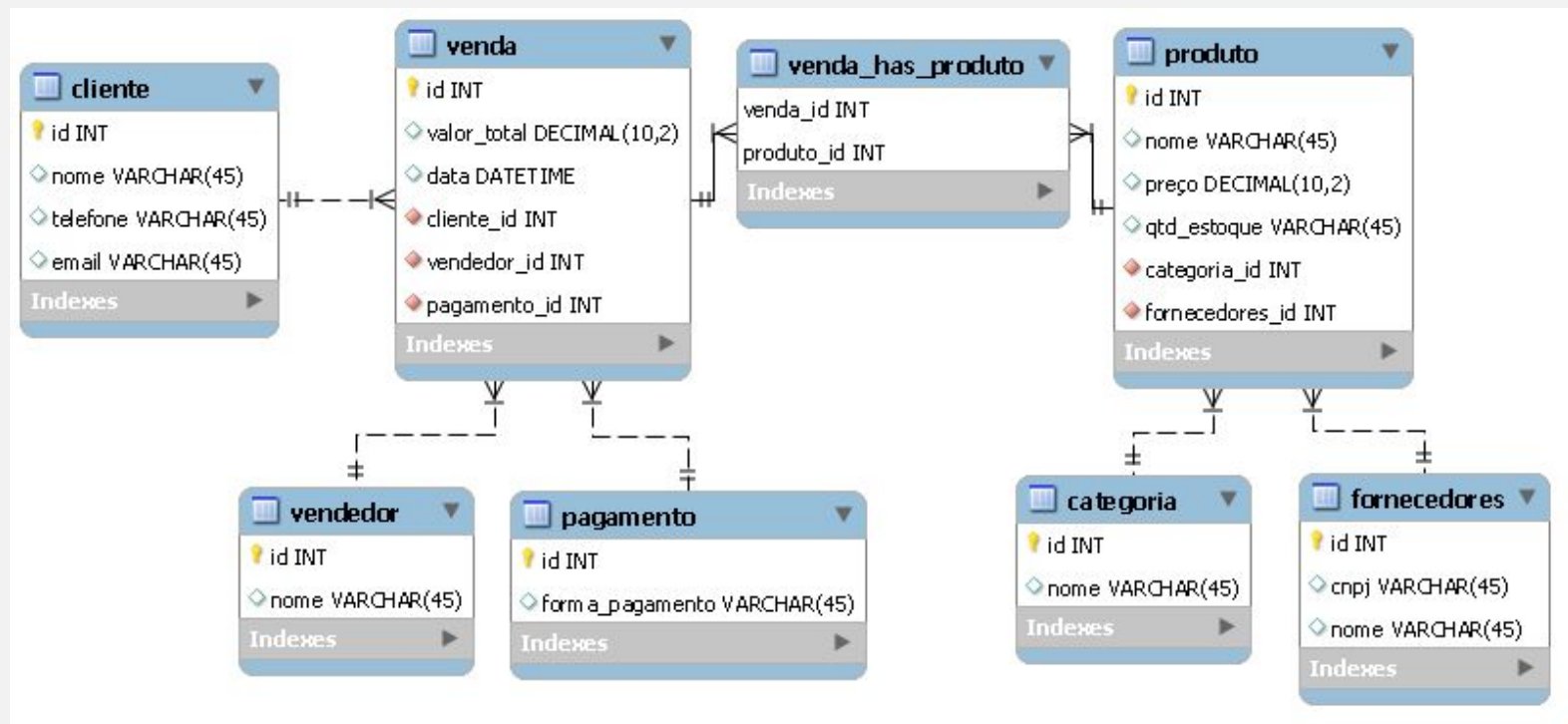
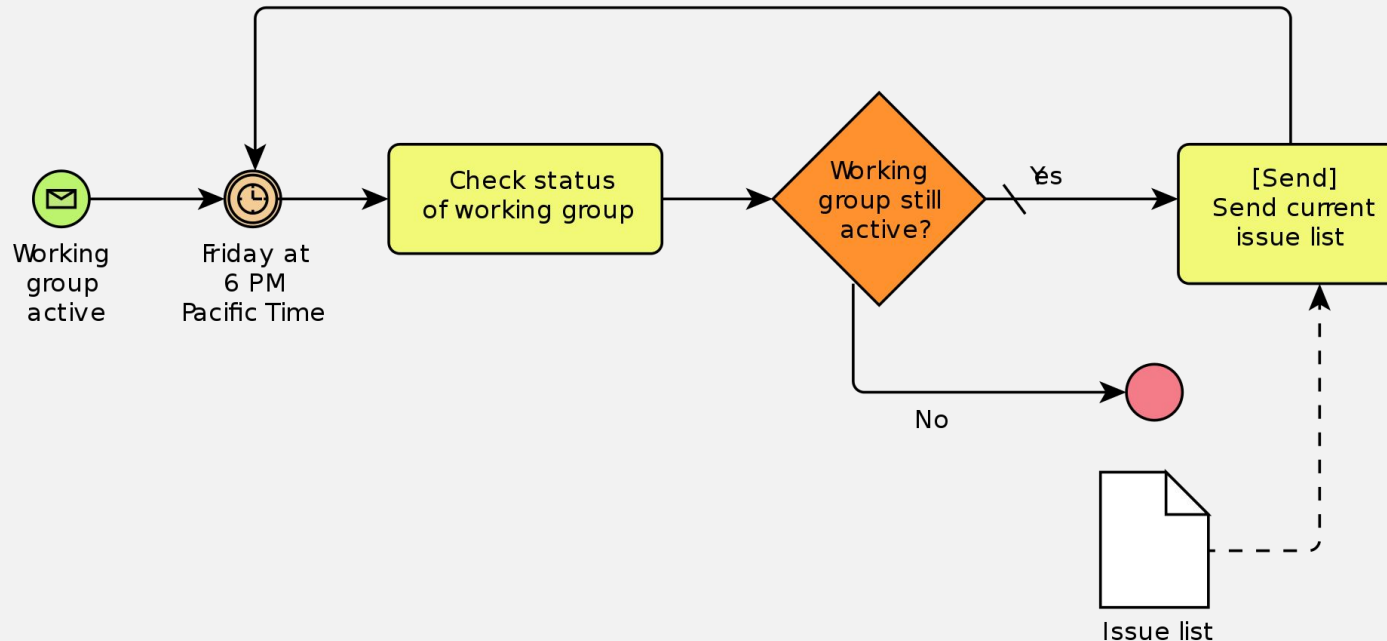


Diagrama de Entidade e Relacionamento



BPMN:

- Notação gráfica padronizada internacionalmente para representar processos de negócio de forma visual e compreensível.



ESPECIFICAÇÃO DE REQUISITOS

Especificação Requisitos

- **Recursos** para Modelagem
 - Recursos de Negócio
 - Processos de Negócio
 - Regras de Negócio

Especificação Requisitos

Recursos do Negócio

- **Recursos Humanos:** funcionários, gerentes, clientes, fornecedores

Recursos Materiais: equipamentos, máquinas, produtos

- **Recursos Informacionais:** dados, documentos, sistemas

Especificação Requisitos

■ Recursos do Negócio

— Exemplo: **Biblioteca**

- Obras
 - Livros
 - Revistas
- Ficha Catalográfica
 - Título
 - Autor
 - Cidade
 - Ano publicação
- Entre outros, ...

Especificação Requisitos

■ Processos do Negócio

- Atividades em que o **estado** dos **recursos** do negócio são **alterados**.
- Um **processo** descreve como um atividade é realizada no negócio.
- Conjunto ordenado de atividades contendo:
 - Começo e Fim bem definidos.
 - Entrada e Saídas bem especificadas.

Especificação Requisitos

■ **Processos do Negócio**

— Exemplo: **Biblioteca**

- Emprestar Livro
- Devolver Livro
- Reservar Obra
- Pesquisar Obra
- Entre outros, ...

Especificação Requisitos

■ Regras do Negócio

- Condições que **definem** ou **restringem** algum aspecto do negócio.
- Representam um conhecimento a respeito do negócio.
- Definem como os processos devem ser executados.

Especificação Requisitos

■ Regras do Negócio

— Exemplo: ***Emprestar Livro da Biblioteca***

- O empréstimo de obras da biblioteca pode ocorrer apenas para usuários previamente cadastrados.
- A devolução de obras após a data limite tem multa de R\$ 1.50 por dia de atraso.

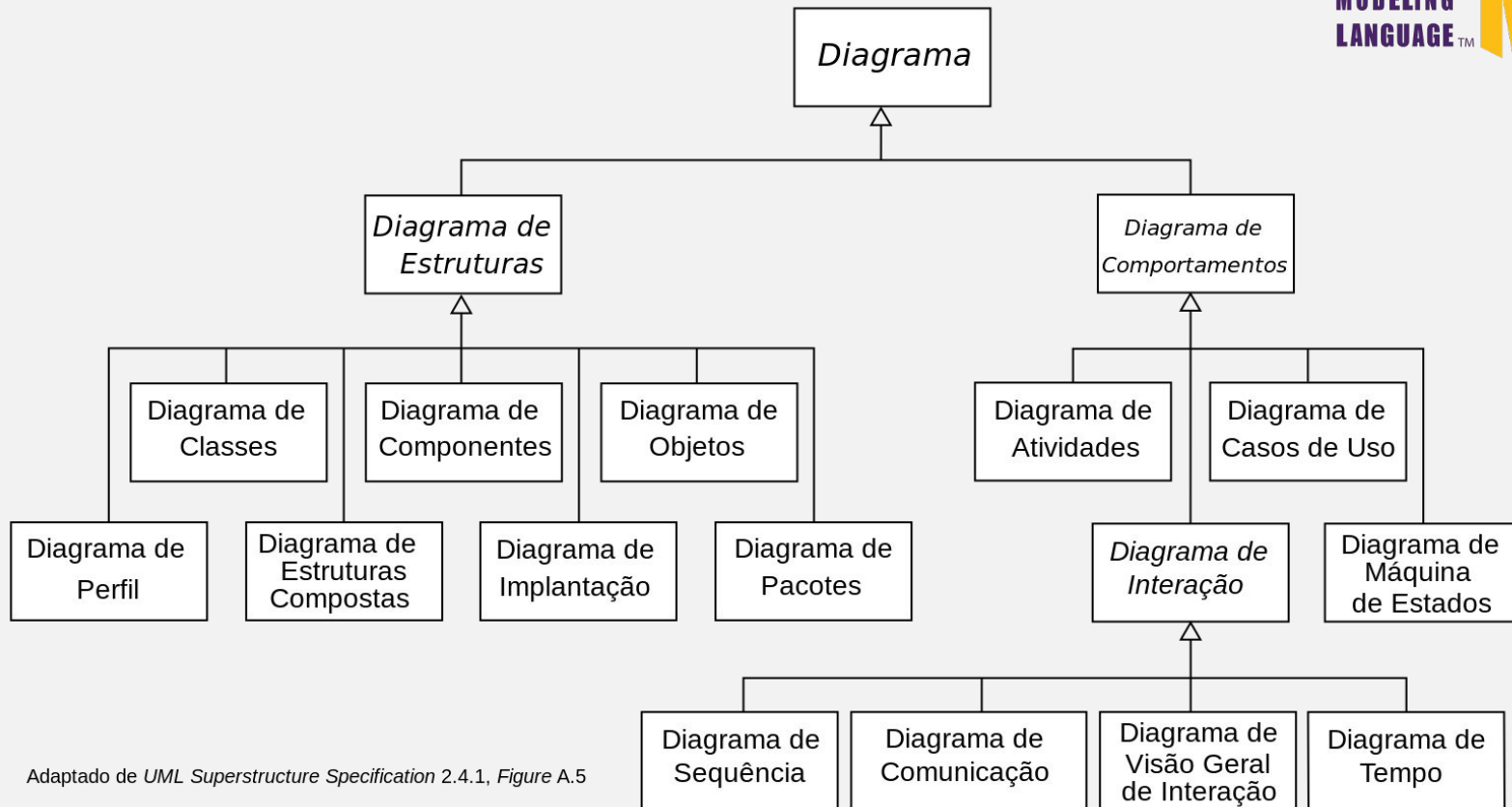
UML

O que é UML?



- Unified Modeling Language.
- Linguagem de Modelagem Unificada.
- Linguagem-padrão para **elaboração da estrutura de projetos de software.**
- Permite que os analistas e desenvolvedores **visualizem os projetos através de diagramas padronizados**

O que é UML?



Adaptado de UML Superstructure Specification 2.4.1, Figure A.5

O que é UML?



Diagramas estruturais

- Existem para visualizar, especificar, construir e documentar os aspectos estáticos de um sistema, ou seja, a representação de seu esqueleto e estruturas.

O que é UML?



Diagramas estruturais

- Diagrama de classes
- Diagrama de objetos
- Diagrama de componentes
- Diagrama de instalação ou de implantação
- Diagrama de pacotes
- Diagrama de estrutura composta
- Diagrama de perfil

O que é UML?



Diagramas comportamentais

- Utilizado para visualizar, especificar, construir e documentar aspectos dinâmicos do sistema.
- Representação das partes que passam por alteração.

O que é UML?



Diagramas comportamentais

- Diagrama de caso de uso
- Diagrama de estados
- Diagrama de atividade
- Diagramas de interação
 - Diagrama de sequência
 - Diagrama Visão Geral de Interação ou de interação
 - Diagrama de colaboração ou comunicação
 - Diagrama de tempo ou temporal

DIAGRAMA DE CASO DE USO

Diagrama de Caso de Uso

- Representação gráfica que **descreve as funcionalidades propostas para o sistema que está sendo projetado.**
- Ferramenta excelente para levantamento dos requisitos funcionais do sistema.
- Um **caso de uso** representa uma unidade discreta de interação entre um ator (humano, dispositivo, ou outro software) e o sistema.

Diagrama de Caso de Uso

■ Caso de Uso de **Negócio**

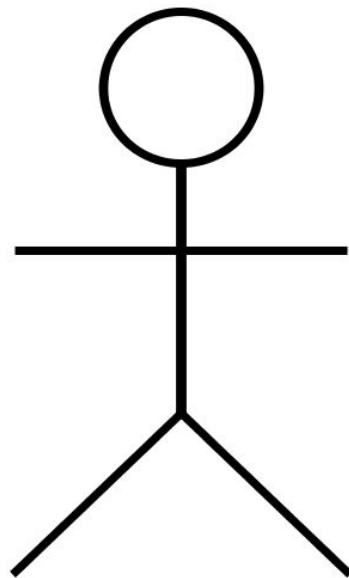
- Descrever como o negócio é usado por seus clientes e parceiros.
- **Visão do Cliente**

■ Caso de Uso de **Sistema**

- Representar os requisitos funcionais dos sistemas.
- **Visão do Desenvolvedor**

Atores

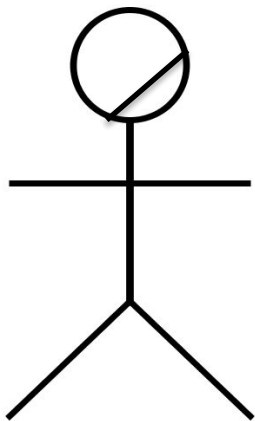
- **Especifica um papel executado** por um usuário, um dispositivo, outro sistema que interage com o assunto (sistema).
- O Ator deve ser externo ao sistema.
- Um ator é **representado por um boneco palito** (*stick man*).



Actor

Atores

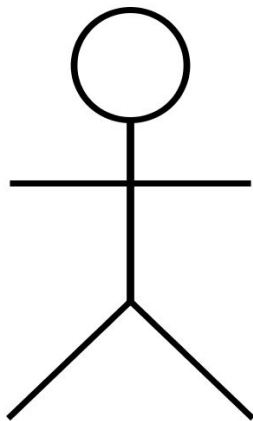
- Um ator pode ser utilizado tanto para representar um papel de negócio quanto de sistema.



Actor

Ator de Negócio

Por exemplo, colaboradores que atuam na empresa.



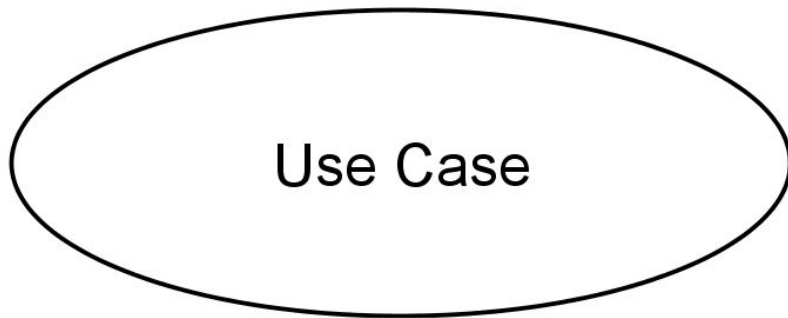
Actor

Ator de Sistema

Por exemplo, usuários do sistema que será desenvolvido.

Caso de Uso

- **Especificação de um conjunto de ações executadas por um sistema, que contém um resultado observável.**
- **Caso de uso é representado por uma elipse, com o nome do caso de uso dentro.**



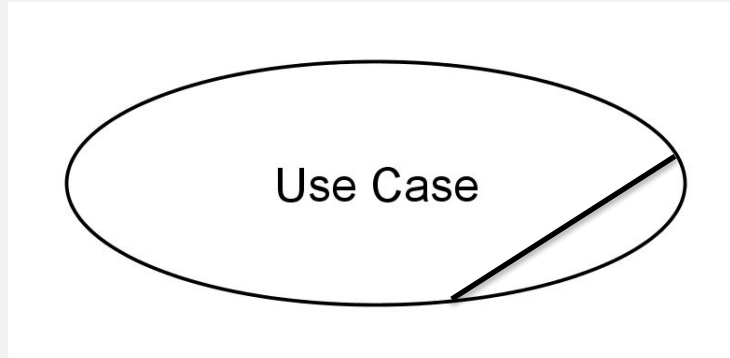
O **nome do caso de uso** deve ser composto por um **VERBO + SUBSTANTIVO** (indicando a ação).

Exemplo:

- Efetuar login
- Cadastrar cliente

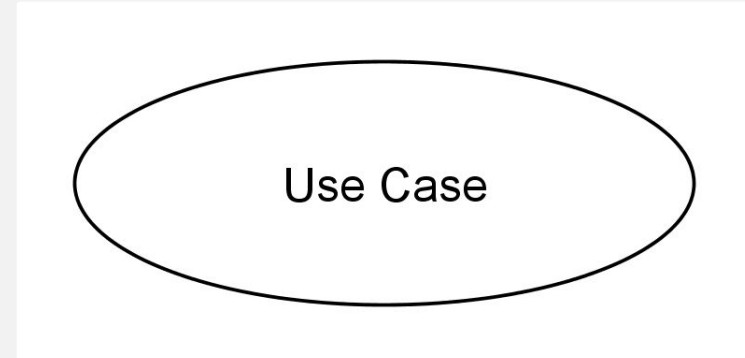
Caso de Uso

- Um caso de uso pode representar tanto uma ação de negócio quanto de sistema.



Caso de Uso de Negócio

Por exemplo, a ação de servir o jantar em um restaurante.



Caso de Uso de Sistema

Por exemplo, a ação de efetuar o login no sistema.

Relacionamentos

- Define relações entre os elementos do diagrama.
- Ajudam a descrever os casos de uso.
- Tipos de Relacionamentos
 - Associação
 - Agregação
 - Composição
 - Herança
 - Dependência

Relacionamentos

Associação

A **associação** representa um **relacionamento estrutural entre duas classes**.

Significa que **objetos de uma classe estão relacionados a objetos de outra classe**.

É o relacionamento **mais comum** em diagramas de classes.

Relacionamentos

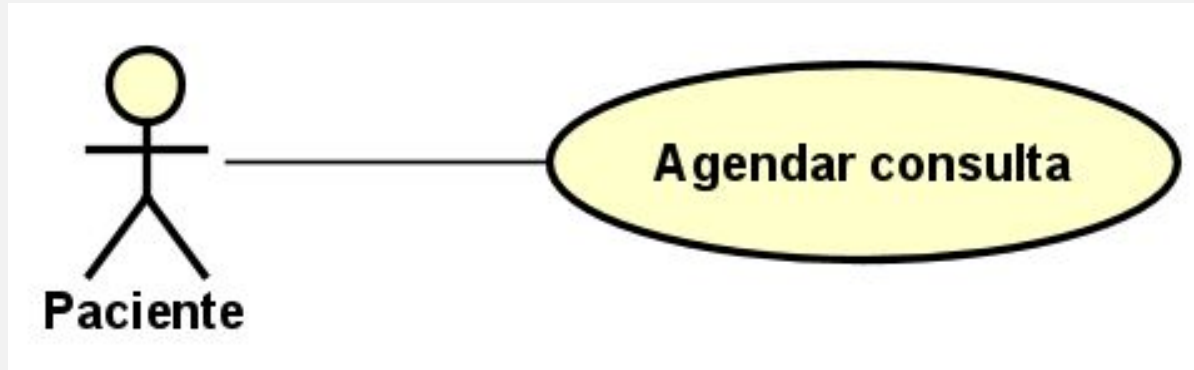
■ Associação entre Ator e Caso de Uso

- Demonstra que o *ator utiliza um caso de uso*.
 - Requisitando a execução de uma funcionalidade
 - Recebendo o resultado de uma funcionalidade
- Representado por uma **reta podendo ser direcionada ou não**.

Relacionamentos

■ Associação entre Ator e Caso de Uso

- A associação é representada por **uma linha simples entre as classes.**

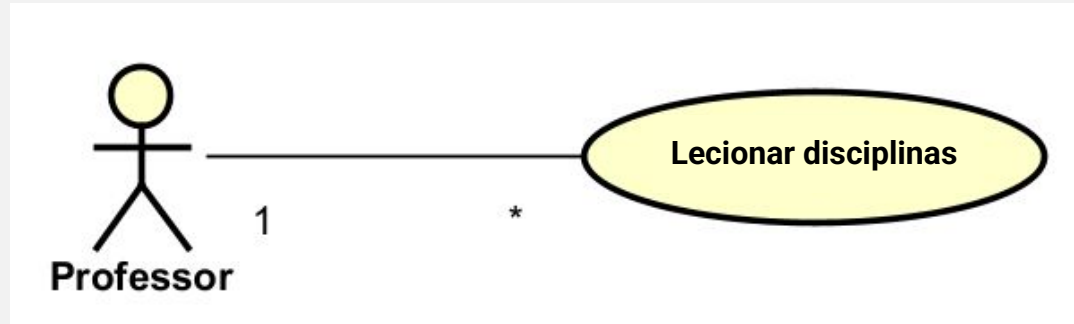


Relacionamentos

Associação com Multiplicidade

A multiplicidade indica **quantos objetos** podem participar da **relação**.

Significado:



- Um professor pode lecionar várias disciplinas
- Cada disciplina tem um professor

Relacionamentos

Associação com Multiplicidade

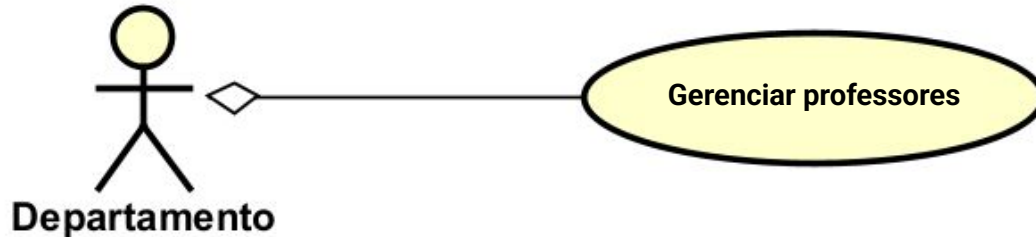
Exemplos de multiplicidade

Notação	Significado
1	exatamente um
0..1	zero ou um
*	muitos
1..*	um ou mais

Relacionamentos

Agregação

A **agregação** representa um relacionamento **todo-parte**, mas com **independência entre os objetos**. Ou seja: As partes **podem existir mesmo sem o todo**.



Significado:

- Um departamento possui professores
- Os professores podem existir independentemente do departamento

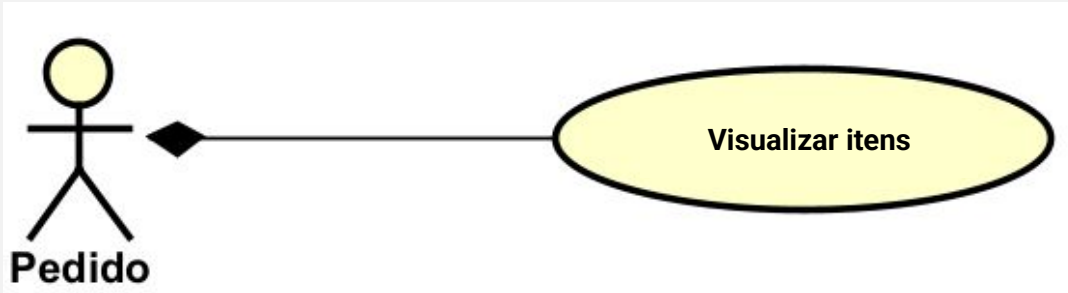
Relacionamentos

Composição

A **composição** também representa uma relação **todo-parte**, porém **mais forte que a agregação**. Na composição: A parte **não pode existir sem o todo**.

Significado:

- Um pedido possui itens
- Um item não existe sem o pedido



Relacionamentos

Diferença entre Agregação e Composição

Característica	Agregação	Composição
Relação	todo-parte	todo-parte
Dependência	fraca	forte
Existência independente	sim	não
Símbolo	losango vazio	losango preenchido

Relacionamentos

Herança

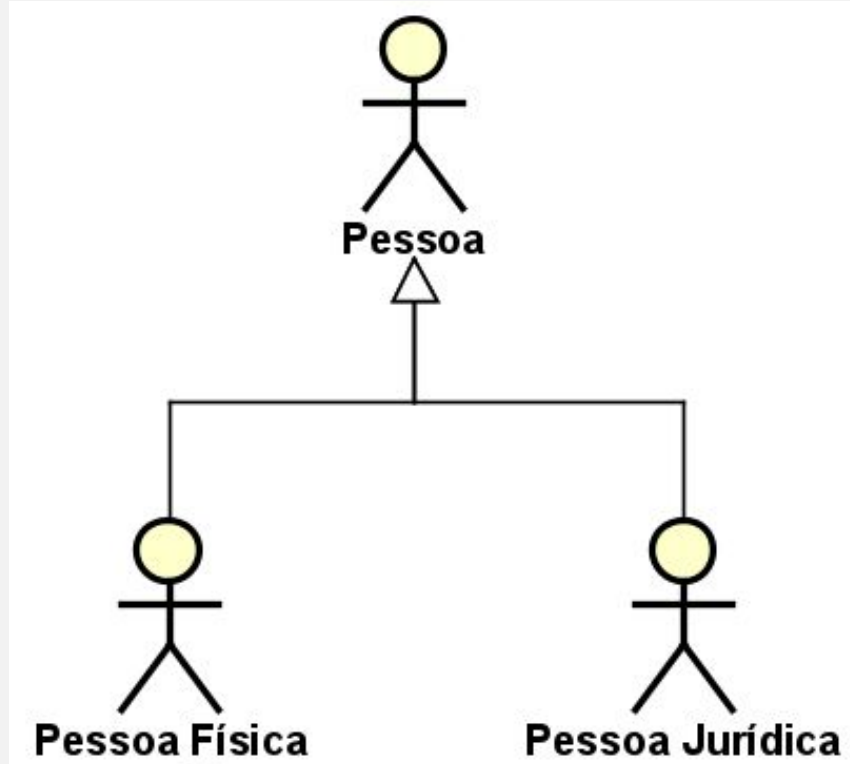
A **herança** representa uma relação **entre uma classe mais geral e outra mais específica**.

A classe filha **herda atributos e métodos da classe pai**.

Esse relacionamento é fundamental na **programação orientada a objetos**.

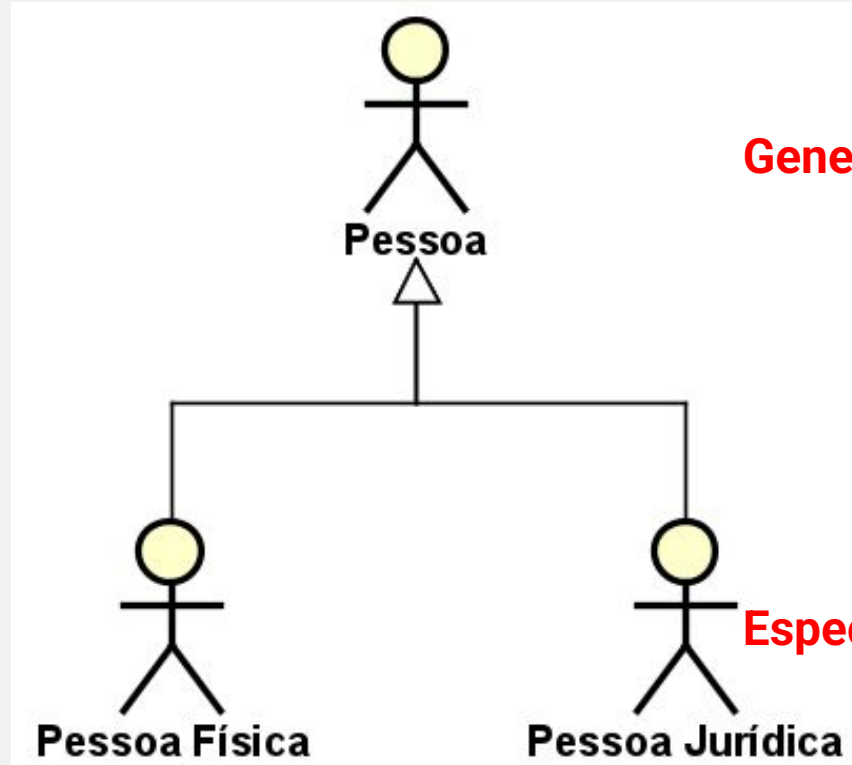
Relacionamentos

■ Generalização



Relacionamentos

■ Generalização

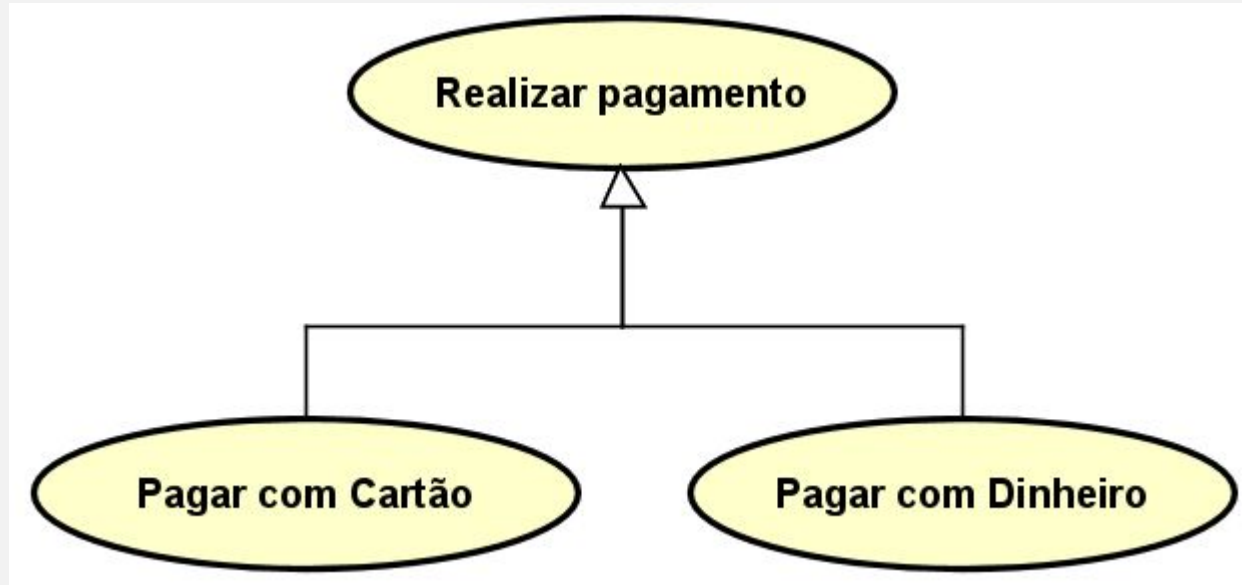


Generalização

Especialização

Relacionamentos

■ Generalização



Relacionamentos

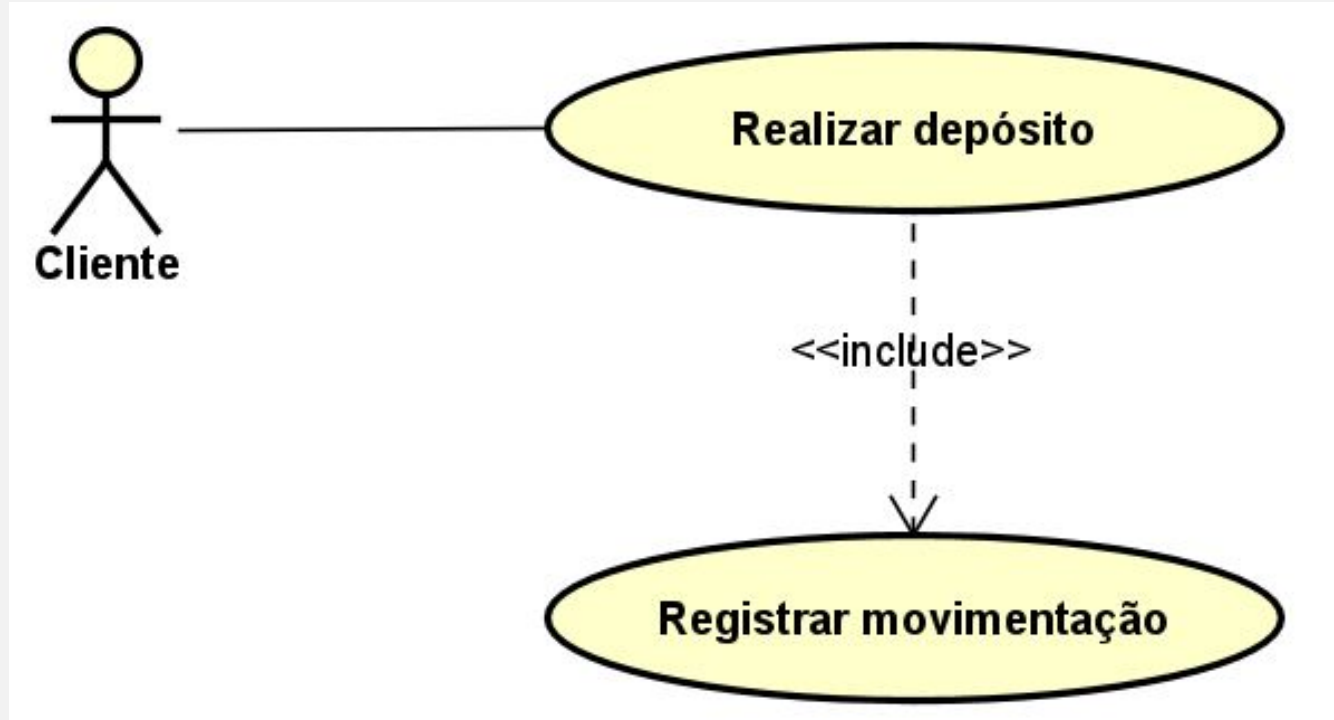
Dependência: Include

O relacionamento **<<include>>** indica que **um caso de uso sempre inclui outro caso de uso como parte de seu comportamento.**

- Um caso de uso **reutiliza obrigatoriamente outro caso de uso.**
- O caso incluído é executado **sempre que o caso principal ocorre.**

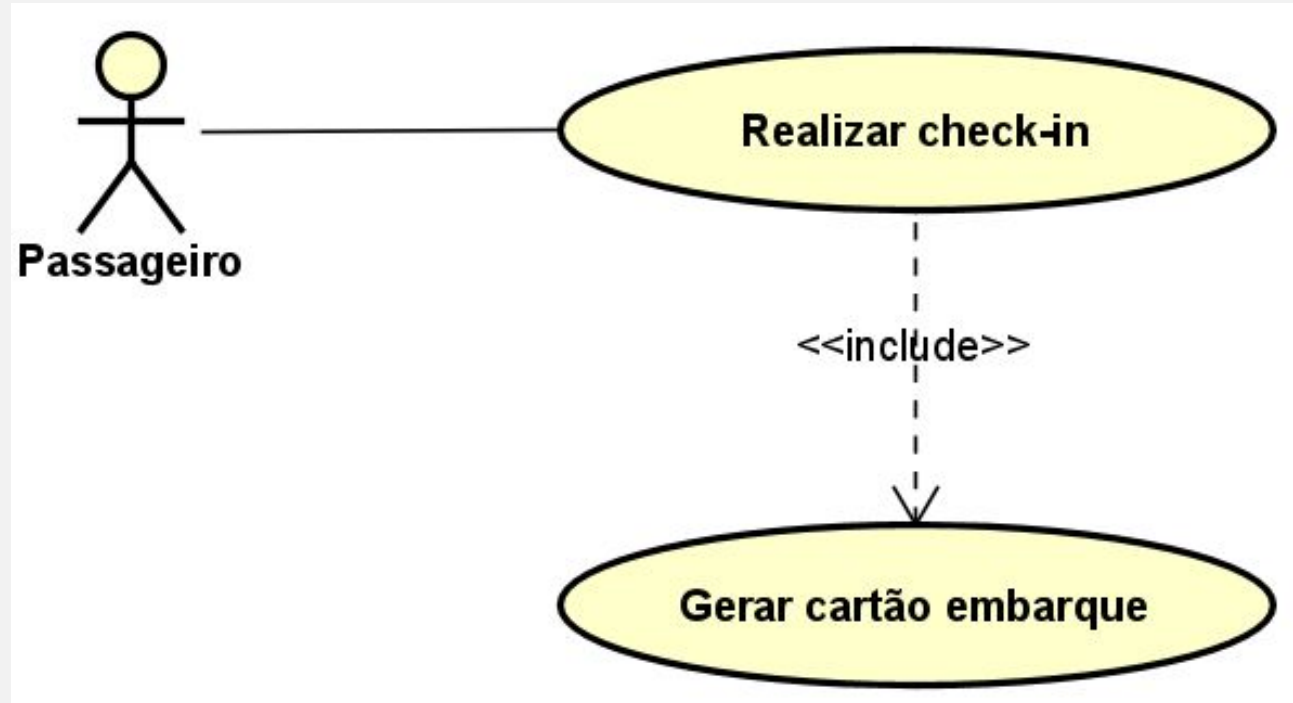
Relacionamentos

■ Include



Relacionamentos

■ Include



Relacionamentos

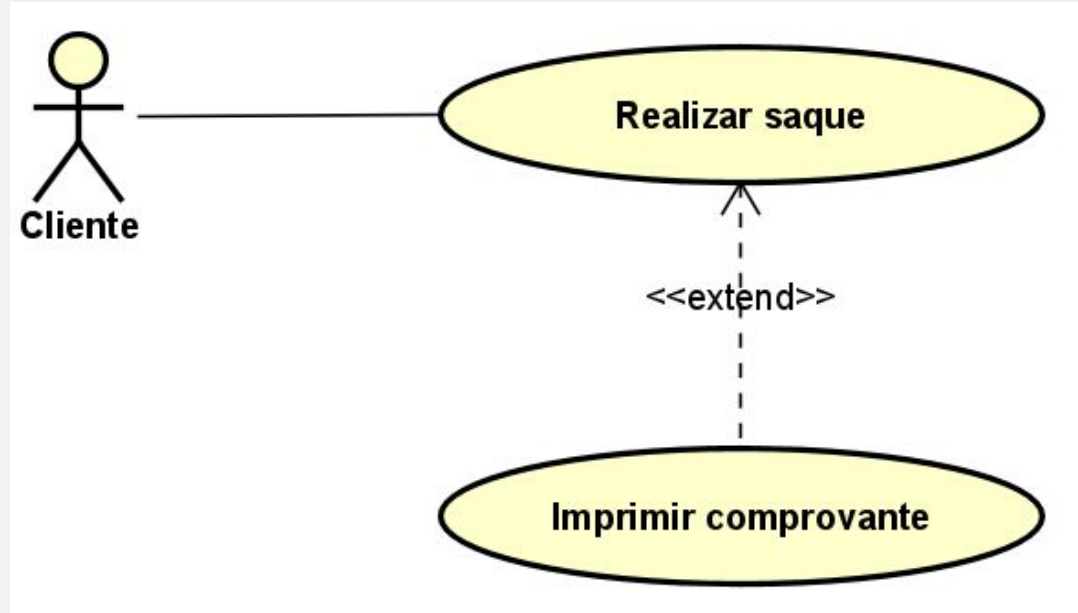
Dependência: *Extend*

O relacionamento **<<extend>>** indica que **um caso de uso pode estender o comportamento de outro em determinadas condições.**

- A execução ocorre **apenas em situações específicas.**
- Diferente do **include**, o **extend** não é obrigatório.

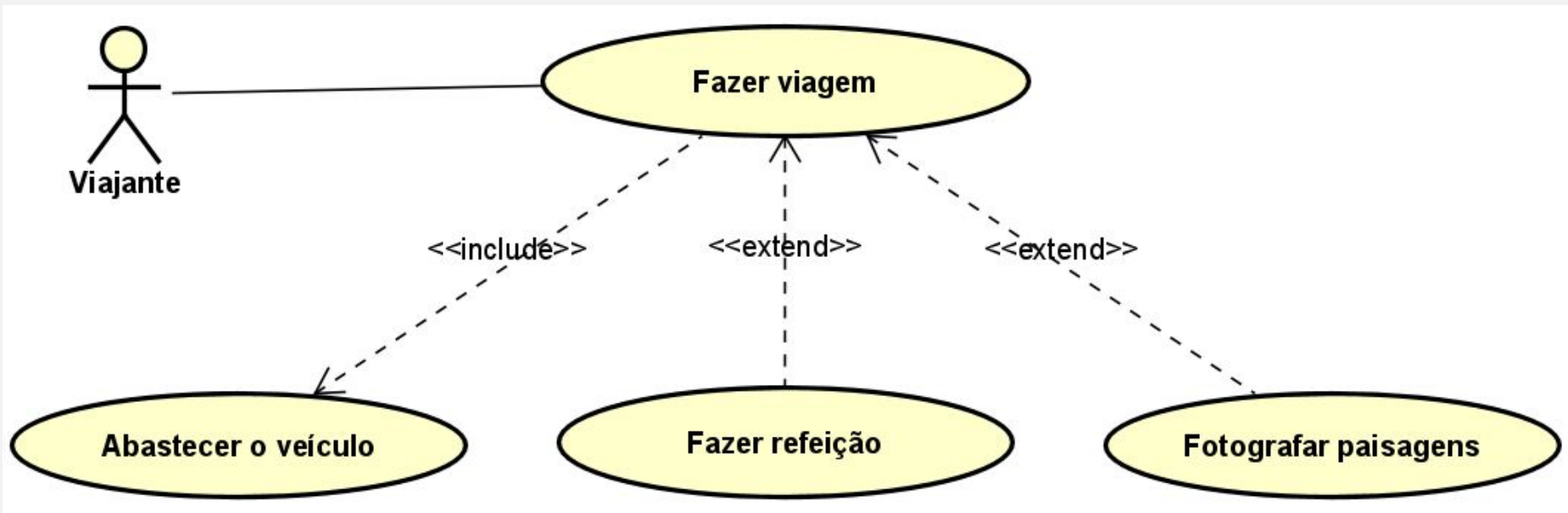
Relacionamientos

■ Extend



Relacionamentos

■ Extend

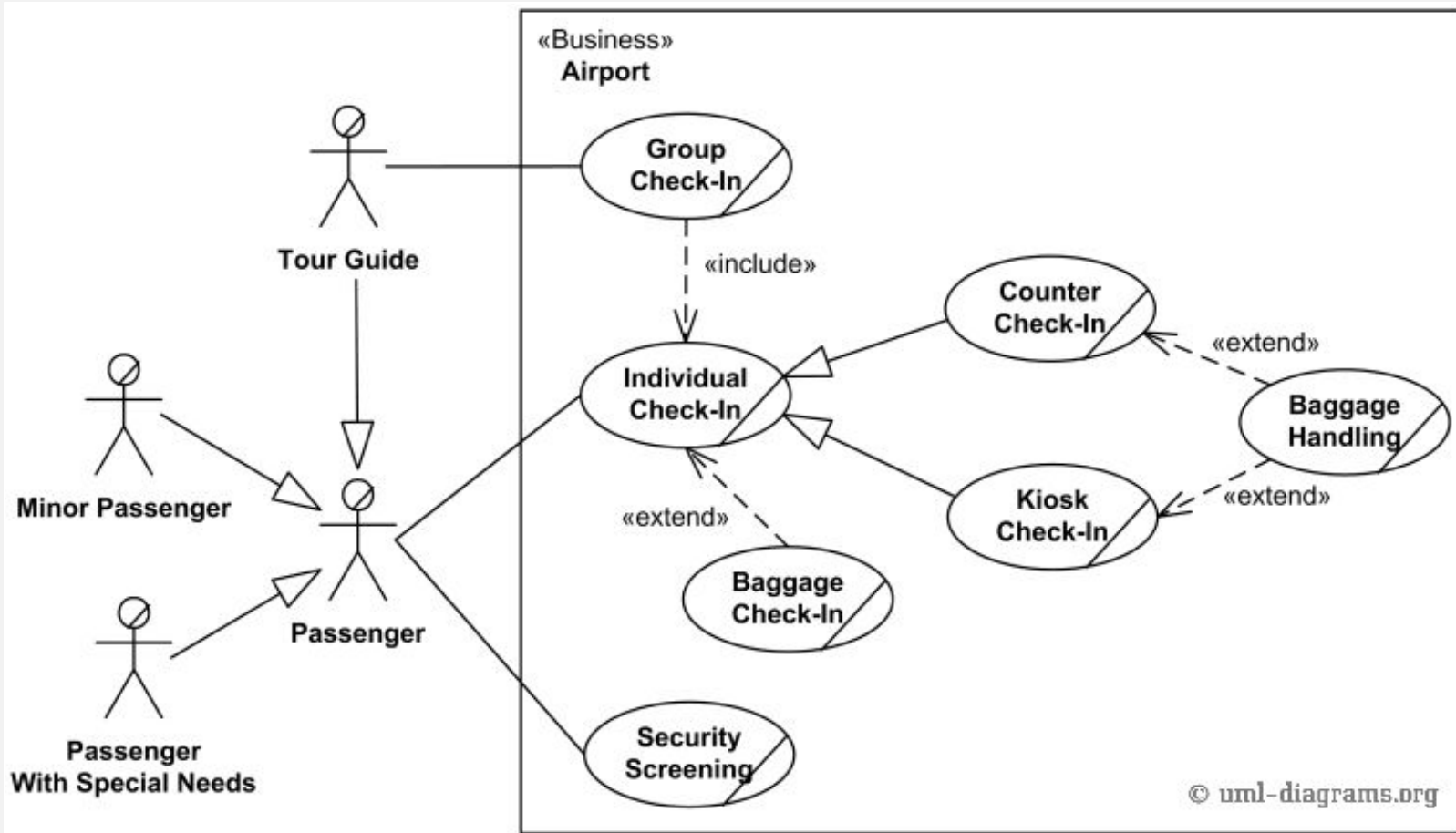


Relacionamentos

Diferença entre <<include>> e <<extend>>

Característica	<<include>>	<<extend>>
Execução	Sempre ocorre	Ocorre apenas em certas condições
Tipo	Obrigatório	Opcional
Objetivo	Reutilização de funcionalidade	Extensão de comportamento
Direção da seta	para o caso incluído	para o caso base

Airport Check-In



Exemplo 1: Sistema de Gerenciamento de Biblioteca

UC1:	Realizar Empréstimo de Livro
Descrição	Este caso de uso permite que o bibliotecário empreste um livro a um usuário.
Ator Principal	Bibliotecário
Pré-condições	O bibliotecário deve estar autenticado no sistema.
Pós-condições	O livro é emprestado ao usuário com sucesso.
Fluxo principal de eventos	1. O sistema exibe a lista de livros disponíveis. 2. O bibliotecário seleciona o livro desejado. 3. O sistema solicita o número do cartão do usuário. 4. O bibliotecário insere o número do cartão do usuário. 5. O sistema registra o empréstimo do livro. 6. O sistema atualiza o status do livro como "emprestado".



Exemplo 1: Sistema de Gerenciamento de Biblioteca

UC2:	Consultar Disponibilidade de Livro
Descrição	Este caso de uso permite que os usuários (clientes) verifiquem a disponibilidade de um livro na biblioteca.
Ator Principal	Usuário
Pré-condições	O usuário deve estar autenticado no sistema.
Pós-condições	O usuário visualiza o status de disponibilidade do livro.
Fluxo principal de eventos	1. O sistema exibe a barra de pesquisa. 2. O usuário insere o título do livro desejado. 3. O sistema verifica a disponibilidade do livro. 4. O sistema exibe o status de disponibilidade do livro.



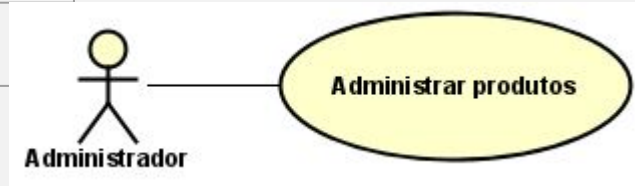
Exemplo 2: Sistema de Compras Online

UC1:	Realizar Compra Online
Descrição	Este caso de uso permite que um cliente compre produtos online.
Ator Principal	Cliente
Pré-condições	O cliente deve estar autenticado no sistema.
Pós-condições	O pedido é registrado no sistema.
Fluxo principal de eventos	1. O cliente procura produtos. 2. O cliente seleciona os produtos desejados. 3. O cliente adiciona os produtos ao carrinho de compras. 4. O cliente revisa o carrinho. 5. O cliente confirma o pedido. 6. O sistema registra o pedido.



Exemplo 2: Sistema de Compras Online

UC2:	Gerenciar Produtos
Descrição	Este caso de uso permite que o administrador gerencie produtos no sistema.
Ator Principal	Administrador
Pré-condições	O administrador deve estar autenticado no sistema.
Pós-condições	As alterações nos produtos são registradas no sistema.
Fluxo principal de eventos	1. O sistema exibe a lista de produtos. 2. O administrador seleciona um produto. 3. O administrador faz alterações no produto. 4. O administrador salva as alterações.





Software gratuito para modelagem UML



astah.net/products/free-student-license/

Get a free student license

Students who have an academic email address are eligible to receive a **FREE license** for Astah UML.

- This is for a single student's personal use only.
- Instructors and teachers cannot use this student license.
- Instructors and teachers should purchase an [academic license](#).

We no longer accept applications with attachment files. This change had to be made due to a high number of dishonest applications. Please ask your teacher to [purchase a site-wide license](#).

By completing this form, you agree that you've read and agree to the Privacy Policy and consent to having Change Vision, Inc collect and store your information.

Academic Email Address*

School name*

First name*

Last name*

☐ I confirm that I entered my information correctly and I am a student at the school I entered above.*

☐ I understand that fraudulent applications will result in permanent ineligibility to obtain a free student license.*

[Apply now](#)

ATIVIDADES PRÁTICAS

Represente as seguintes relações com UML



1. Associação (Conexão Simples)

Cenário: Em uma plataforma de e-commerce, um **Usuário Cliente** precisa interagir com o catálogo.

- **Tarefa:** Represente, utilizando uma associação, a relação entre o **Usuário Cliente** e a ação **Consultar Produto**.

Represente as seguintes relações com UML



2. Agregação (Todo-Parte Fraco)

Cenário: Em um sistema de gerenciamento de repositórios (como o GitHub), um **Perfil Organizacional** agrupa diversos **Repositórios**. Note que um repositório pode ser transferido para outra organização ou existir de forma independente.

- **Tarefa:** Modele a relação entre o **Usuário Organização** e o **Gerenciar Repositório**, utilizando agregação para indicar que um compõe o outro, mas com ciclos de vida independentes.

Represente as seguintes relações com UML



3. Composição (Todo-Parte Forte)

Cenário: Em um software de edição de texto, um **Documento** é formado por várias **Páginas**. Se o documento for excluído permanentemente, todas as suas páginas associadas também serão eliminadas do sistema.

Tarefa: Represente a relação entre o **Usuário Editor** (responsável pelo documento) e a estrutura interna, modelando a composição entre **Estruturar Documento** e **Estruturar Página**.

Represente as seguintes relações com UML

4. Herança / Generalização (Especialização)

Cenário: Um sistema de segurança possui diferentes tipos de contas. Tanto o **Usuário Administrador** quanto o **Usuário Comum** possuem um login e uma senha, mas o administrador tem permissões extras.

- **Tarefa:** Projete um relacionamento entre **Usuário**, **Usuário Comum** e **Usuário Administrador**.

Represente as seguintes relações com UML



5. Include (Comportamento Obrigatório)

Cenário: No módulo de finanças de um ERP, sempre que um **Usuário Contador** decide **Fechar Folha**, o sistema deve, obrigatoriamente, executar uma rotina interna para **Gerar Log**.

- **Tarefa:** No diagrama de casos de uso, utilize o relacionamento `<<include>>` para mostrar que a ação **Fechar Folha** inclui necessariamente a ação **Gerar Log**.

Represente as seguintes relações com UML



6. Extend (Comportamento Opcional/Condicional)

Cenário: Em um sistema de streaming, ao executar a ação **Assistir Vídeo**, o **Usuário Assinante** pode decidir, caso deseje, **Habilitar Legendas**.

- **Tarefa:** Representar a relação entre os casos de uso utilizando `<<extend>>`, indicando que **Habilitar Legendas** é uma extensão opcional do fluxo principal.

Descrição: A biblioteca central da universidade deseja modernizar seu sistema. O bibliotecário deve poder adicionar novos livros ao acervo, remover livros danificados e registrar empréstimos/devoluções. Os alunos e professores (usuários) podem buscar livros no catálogo e solicitar empréstimos. O sistema deve notificar os usuários por e-mail quando um livro reservado estiver disponível ou quando o prazo de entrega estiver próximo.

Tarefas:

1. Identifique todos os **Atores**.
2. Identifique os **Casos de Uso** principais.
3. Modele o diagrama no Astah, representando os relacionamentos de **Associação** e pelo menos um relacionamento de **Inclusão** (ex: **Registrar Empréstimo** pode *incluir* **Verificar Disponibilidade**).

Descrição: Um aplicativo conecta clientes a restaurantes. O **Cliente** pode buscar restaurantes, visualizar cardápios, fazer um pedido (que inclui pagamento) e rastrear a entrega. O **Entregador** recebe notificações de novos pedidos, visualiza a rota e confirma a entrega. O **Restaurante** recebe pedidos, atualiza o status (preparando, pronto) e o cardápio. O pagamento é sempre processado por um **Gateway de Pagamento** externo. O pedido pode ser cancelado pelo cliente antes da confirmação do restaurante.

Tarefas:

1. Utilize o relacionamento de **Extensão** para representar a opção de "**Cancelar Pedido**". O caso de uso **Fazer Pedido** é estendido por **Cancelar Pedido** em um ponto de extensão chamado "**antes da confirmação**".
2. Utilize **Inclusão** para mostrar que **Fazer Pedido** *precisa* incluir a funcionalidade **Processar Pagamento**.
3. Represente o ator externo **Gateway de Pagamento**.

Sistema de Gestão de Consultório Médico

Exercício 3



Descrição: Um consultório médico precisa de um sistema para agilizar seus processos. A **Recepcionista** é responsável por marcar consultas, cancelar consultas e gerar lembretes de confirmação para os pacientes via SMS. O **Médico** precisa acessar sua agenda diária, visualizar o prontuário do paciente (que inclui histórico e alergias) e registrar as anotações da consulta. O **Paciente** (ator externo) pode visualizar suas próprias informações de agendamento através de um portal online. O sistema deve se integrar a um **Sistema de Envio de SMS** externo para comunicação. O cancelamento de uma consulta só é permitido se feito com pelo menos 24 horas de antecedência.

Tarefas:

1. Identifique todos os **Atores**.
2. Identifique os **Casos de Uso** principais.
3. Utilize um relacionamento de Extensão para representar "**Cancelar Consulta**". Crie um *Extension Point* no caso de uso **Marcar Consulta** chamado "**horario_antecedencia_24h**" para indicar a condição de sucesso.
4. Utilize Inclusão para mostrar que **Registrar Anotações da Consulta** precisa consultar o **Visualizar Prontuário**.
5. Represente o ator externo **Sistema de Envio de SMS**.

Descrição: Uma plataforma de cursos online permite que **Alunos** se matriculem em cursos, assistam a videoaulas, realizem avaliações online e baixem materiais complementares. O **Instrutor** é responsável por publicar videoaulas, upload de materiais, criar questões para as avaliações e emitir certificados para alunos aprovados. O **Administrador** da plataforma gerencia os usuários (instrutores e alunos) e os cursos disponíveis. A plataforma possui um **Sistema de Anti-fraude** integrado que, durante a realização de uma avaliação, pode acionar uma Análise de Suspeita de Cola se detectar comportamento incomum (ex: o aluno mudar de aba no navegador). A aprovação e emissão do certificado só ocorre se o aluno for aprovado na avaliação.

Tarefas:

1. Utilize o relacionamento de Extensão para representar a funcionalidade de "Análise de Suspeita de Cola". Crie um Extension Point no caso de uso Realizar Avaliação Online para indicar quando a extensão ocorre (ex: comportamento_incomum_detectado).
2. Utilize Inclusão para mostrar que Emitir Certificado *depende* que o aluno tenha sido Aprovado na Avaliação.
3. Represente o ator externo Sistema de Anti-fraude.

FIM